

Curso Inicial Front-End

Clase 03: Git

Avanzado: Ramas y Colaboración

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)



Presentación

Una vez que dominas el flujo básico, es hora de trabajar en paralelo. En esta clase descubriremos las **Ramas (Branches)**, la funcionalidad que permite crear experimentos y nuevas funciones sin romper la versión estable de tu web. Aprenderemos a fusionar trabajos y, lo más importante, a resolver **Conflictos** como un ingeniero senior.



Objetivos

- Dominar la creación y el cambio entre ramas (`git branch`, `git checkout`).
- Realizar fusiones (`merges`) y entender el flujo de trabajo en equipo.
- Detectar, entender y resolver conflictos de código de forma manual en VS Code.



Bloques Temáticos

- Tema 1: Gestión de ramas (Branches), Merge y resolución de conflictos.
 - ¿Qué es un Branch (Rama)?
 - Gestión de Ramas y Comandos
 - Unir Ramas (Merge)
 - Conflictos de Fusión (Merge Conflicts)
 - Ignorando Archivos y El Limbo (Stash)
- Tema 2: Trabajo colaborativo y publicación de sitios en GitHub Pages
 - Flujo de Trabajo Corporativo: Fork y Pull Request (PR)

En la clase anterior aprendimos a tomar "fotografías" (commits) de nuestro código para tener un historial seguro, y a subirlas a la nube de [GitHub](https://github.com). Pero, ¿qué sucede cuando trabajamos en equipo? Si tres personas editan el mismo archivo al mismo tiempo, el historial chocaría. O peor aún, ¿qué pasa si quieres probar una idea loca en tu código pero te da terror romper la versión que ya funciona?

Para solucionar estos problemas de la vida real, hoy aprenderemos a trabajar como verdaderos profesionales usando **Ramas (Branches)**. Esto nos permitirá crear "universos paralelos" de nuestro proyecto.

Tema 1: Gestión de ramas (Branches), Merge y resolución de conflictos.

1.1. ¿Qué es un Branch (Rama)?

Imagina que el proyecto es una película. La rama `main` es la versión final impecable que llega al cine. Si como director quieres probar un final alternativo o grabar una escena arriesgada, no destruyes la película original; creas una copia exacta en un universo paralelo. A esa copia le llamamos **Rama**.

- **Branching (Ramificación):** Es aislar tu trabajo. Todo el código que escribas, modifiques o rompas en tu rama no afectará a la versión principal (`main`) hasta que tú decidas unir las conscientemente.
- **¿Qué es una rama técnicamente?** Al contrario de lo que parece, una rama no es una copia gigante de tu carpeta (no duplica el peso del proyecto). Es simplemente un **puntero o flecha invisible** que apunta a un commit específico. ¡**Git** es tan eficiente que crear una rama toma milisegundos!
- **HEAD (Tú estás aquí):** Imagina que el "HEAD" es tu ubicación en el mapa de GPS de **Git**. Es un puntero especial que le dice a la computadora en qué rama y en qué commit exacto estás parado en este momento. Moverse de rama es simplemente decirle al HEAD: "*Llévame a la otra línea temporal*".
 - **¡Pánico! Detached HEAD (Cabeza Desconectada):** Si usas `git checkout <hash-de-un-commit-viejo>`, el HEAD se despega de la rama y

apunta directamente a una foto del pasado. No entres en pánico, no rompiste nada, solo estás "mirando el museo". Para volver al presente, escribe `git switch main`.

 **Recurso Oficial:** [Git - Branching and Merging](#)

1.2. Gestión de Ramas y Comandos

Abre tu **Git** Bash en la carpeta de tu proyecto. Aquí tienes los comandos vitales para viajar entre universos.

En versiones recientes de **Git**, se introdujo el comando `switch` porque es más fácil de entender que el viejo `checkout`. Puedes usar cualquiera de los dos, pero te recomendamos el moderno.

Acción	Comando Tradicional	Comando Moderno (Recomendado)
Crear una rama nueva	<code>git branch <nombre></code>	<code>git branch <nombre></code>
Moverse a otra rama	<code>git checkout <nombre></code>	<code>git switch <nombre></code>
Crear y moverse de una vez	<code>git checkout -b <nombre></code>	<code>git switch -c <nombre></code>

Puesta en marcha visual: Si usas Visual Studio Code, mira la esquina inferior izquierda de tu pantalla (en la barra azul). Ahí verás escrito el nombre de la rama en la que estás actualmente (por ejemplo, `main`). ¡Si haces clic ahí, puedes cambiar de rama sin usar la consola!

Convención de Nombres

En las empresas serias no llamamos a las ramas "rama-de-juan" o "probando-cositas". Para mantener el orden en equipos grandes, usamos prefijos estandarizados:

- `feature/login`: Para nuevas funcionalidades (ej: programar el formulario de inicio de sesión).
- `bugfix/menu`: Para arreglar errores menores que alguien reportó.
- `hotfix/caida-pago`: Para arreglos de emergencia extremo que deben ir a producción (la web en vivo) inmediatamente.

1.3. Unir Ramas (Merge)

Cuando terminas tu tarea en la rama `feature/login` y estás completamente seguro de que funciona a la perfección, tu objetivo es traer esos cambios geniales de vuelta a la realidad principal (`main`). A este proceso de unir dos líneas temporales lo llamamos **Merge (Fusión)**.

Paso a paso para una fusión exitosa: 1. Asegúrate de haber hecho un `commit` en tu rama actual.

1. Cámbiate a la rama *receptora* (la que va a absorber los cambios): `git switch main`
2. Pídele a **Git** que traiga los cambios de tu otra rama hacia aquí: `git merge feature/login`

Eventualmente escucharás a otros programadores discutir sobre "Merge" y "Rebase". Ambos comandos logran el mismo objetivo final (unir código), pero lo hacen con filosofías opuestas:

Característica	git merge (Fusión Clásica)	git rebase (Reescribir Historia)
¿Cómo actúa?	Busca el punto donde las ramas se separaron y crea un nuevo commit de unión (Merge Commit).	Desconecta tus commits recientes, actualiza la rama base, y vuelve a "pegar" tus commits encima.
Historial Visual	Queda como un árbol con múltiples ramas cruzándose (es el historial real).	Queda como una línea recta perfecta e irreal (es un historial reescrito).
Riesgo	Cero riesgo. Es totalmente destructivo y seguro.	Alto riesgo. Destruye los IDs (Hashes) originales de tus commits.
Regla de Oro	Úsalo siempre como tu herramienta principal para unir código.	NUNCA hagas rebase en una rama pública o que compartas con otros. Solo úsalo en tu rama local súper privada.

1.4. Conflictos de Fusión (Merge Conflicts)

La palabra "Conflicto" da miedo, pero es algo normal de todos los días. Un conflicto ocurre cuando **dos personas modifican exactamente la misma línea del mismo archivo**. **Git** es inteligente, pero no puede leer la mente: no sabe si debe quedarse con el código de tu compañero o con el tuyo. Entonces, **Git** frena la fusión y te pide ayuda humana.

Cuando intentes hacer un **merge** y falle por conflicto, abre el archivo en rojo en Visual Studio Code. Verás que **Git** inyectó unas extrañas marcas de advertencia:

```
<<<<<<< HEAD
<h1>Bienvenidos a la tienda (Diseño de Juan)</h1>
```

```
=====
<h1>Ofertas de Verano (Diseño de Ana)</h1>
>>>>>> feature/ofertas
```

- **HEAD (Current Change):** Lo que ya estaba en la rama donde estás parado (tu código actual).
- **La otra rama (Incoming Change):** Lo que viene de la rama que quieres absorber (el código entrante).
- **La Solución:** ¡No entres en pánico! VS Code te mostrará botones arriba del conflicto que dicen "Accept Current Change" o "Accept Incoming Change". O simplemente puedes borrar manualmente las líneas extrañas (`<<<`, `===`, `>>>`) y escribir el código final perfecto que combine ambas ideas. Finalmente, guarda el archivo, haz `git add .` y luego `git commit` para sellar la paz.

1.5. Ignorando Archivos y El Limbo (Stash)

.gitignore: Lo que NO debe subirse a la Nube

Hay cosas que jamás deben viajar por internet hacia [GitHub](#): archivos con contraseñas secretas, fotos pesadísimas de prueba, o carpetas gigantes autogeneradas por herramientas (como la famosa carpeta `node_modules/` que veremos en [React](#)).

- **La Solución:** Crea un archivo de texto plano en la raíz de tu proyecto y llámalo exactamente `.gitignore` (con un punto al inicio). Dentro de él,

escribe los nombres de las carpetas o archivos. **Git** hará de cuenta que esos archivos son invisibles y jamás los subirá a la bandeja.

El Salvavidas Profesional: `git stash`

Imagina este escenario: Estás trabajando en una rama haciendo una nueva funcionalidad. El código está todo roto y a medias. De repente, tu jefe te pide que cambies de rama urgentemente a `main` para arreglar un error crítico de la web. Problema: No puedes cambiar de rama si tu "Heladera" está llena de código modificado sin guardar. Y tampoco quieres hacer un `commit` basura de algo que no funciona.

Aquí es donde entra el "Cajón Secreto" (Stash):

1. **Guardar en el limbo:** Guarda tus cambios sucios temporalmente en una gaveta secreta para dejar tu espacio de trabajo totalmente limpio: `git stash`
2. **Cambiar y salvar el día:** Ahora tu entorno está limpio. Muévete a la rama urgente (`git switch main`), soluciona el problema de tu jefe y vuelve a tu rama original.
3. **Recuperar del limbo:** Abre la gaveta secreta, saca tus cambios incompletos y vuelve a esparcirlos en tu código exactamente como los dejaste: `git stash pop`

Tema 2: Trabajo colaborativo y publicación de sitios en GitHub Pages

2.1. Flujo de Trabajo Corporativo: Fork y Pull Request (PR)

En un entorno laboral, como empleado junior o semi-senior, **casi nunca** tendrás los permisos de administrador para hacer `git push` directamente a la rama `main` del servidor de la empresa. Hacerlo podría romper la página web en vivo. El flujo real y seguro es este:

1. **Fork (Bifurcación):** Entrás a [GitHub](#) y haces una copia completa e independiente del repositorio sagrado de la empresa hacia tu propia cuenta de [GitHub](#) personal.
2. **Clonar y Trabajar:** Descargas (`git clone`) tu *Fork* a tu computadora, creas tu rama de trabajo (`feature/tarea`), programas y subes (`push`) ese código a tu propia cuenta en la nube.
3. **Pull Request (PR):** Vas a [GitHub](#) y envías una solicitud formal a los administradores del código de la empresa: *"Por favor (Request), jalen (Pull) mi código y únanlo al proyecto principal"*.
4. **Code Review:** Tus compañeros revisan el código de tu PR. Te dejarán comentarios (ej: "Mejora el nombre de esta variable"). Tú corriges localmente, haces otro `push` y, cuando todos aprueban (Approve), un Líder Técnico presiona el botón verde de `Merge`.

Documentación Oficial (Referencias)

- **Git:** <https://git-scm.com/doc>
- **GitHub:** <https://docs.github.com/es>
- **React:** <https://es.react.dev/>